

МИНИСТЕРСТВО ЦИФРОВОГО РАЗВИТИЯ, СВЯЗИ И МАССОВЫХ
КОММУНИКАЦИЙ
РОССИЙСКОЙ ФЕДЕРАЦИИ

Федеральное государственное бюджетное образовательное учреждение
высшего образования
«Сибирский государственный университет телекоммуникаций и
информатики» (СибГУТИ)

Кафедра телекоммуникационных систем и вычислительных средств
(ТС и ВС)

ОТЧЁТ
по лабораторной работе № 1
по дисциплине
«Основы систем мобильной связи» (ОСМС)

по теме:
ВРЕМЕННАЯ И ЧАСТОТНАЯ ФОРМЫ СИГНАЛОВ.
ПРЕОБРАЗОВАНИЯ ФУРЬЕ. ДИСКРЕТИЗАЦИЯ СИГНАЛОВ

Вариант № 23

Студент:
Группа № ИКС-433

А.О. Стаценко

Проверила:
Заведующая кафедрой ТС и ВС

В.Г. Дроздова

Новосибирск 2026 г.

СОДЕРЖАНИЕ

1	ВЫПОЛНЕНИЕ ЛАБОРАТОРНОЙ РАБОТЫ № 1	3
1.1	Цель и задачи работы	3
1.2	Краткие теоретические сведения	4
1.3	Ход выполнения лабораторной работы	6
1.3.1	Пункт 1. Визуализация непрерывного гармонического сигнала (Вариант 23).....	6
1.3.2	Пункт 2. Определение максимальной частоты в спектре сигнала	7
1.3.3	Пункт 3. Выбор частоты дискретизации (теорема Котельникова)	8
1.3.4	Пункт 4. Оцифровка сигнала	8
1.3.5	Пункт 5. Прямое дискретное преобразование Фурье и расчет памяти	9
1.3.6	Пункт 6. Цифро-аналоговое преобразование (восстановление при $f_s = 130$ Гц)	10
1.3.7	Пункт 7. Влияние частоты дискретизации на ЦАП (увеличение в 4 раза)	11
1.3.8	Пункт 8. Спектр голоса в программе Audacity	13
1.3.9	Пункт 9. Чтение аудиофайла в среде MATLAB	14
1.3.10	Пункт 10. Расчет частоты дискретизации аудиофайла на цифровом носителе	14
1.3.11	Пункт 11. Искажение сигнала (прореживание) и воспроизведение	15
1.3.12	Пункт 12. Сравнение спектров оригинального и искаженного сигнала (алиасинг)	15
1.3.13	Пункт 13. Исследование влияния разрядности АЦП на спектр сигнала	17
2	ОТВЕТЫ НА КОНТРОЛЬНЫЕ ВОПРОСЫ.....	20
	ЗАКЛЮЧЕНИЕ	22
	ПРИЛОЖЕНИЕ А. ИСХОДНЫЙ КОД ПРОГРАММЫ В СРЕДЕ MATLAB	23
	ПРИЛОЖЕНИЕ Б. ССЫЛКА НА ЭЛЕКТРОННЫЕ РЕСУРСЫ	28

1 ВЫПОЛНЕНИЕ ЛАБОРАТОРНОЙ РАБОТЫ № 1

1.1 Цель и задачи работы

Цель работы: Получить представление о формах радиосигналов, их частотном и временном представлении, а также о преобразованиях Фурье и аналоговоцифровых преобразованиях сигналов, частоте дискретизации сигналов.

Задачи практической работы (программа исследований):

1. Сгенерировать и визуализировать непрерывный периодический сигнал в соответствии с индивидуальным вариантом 23.
2. Аналитически определить максимальную частоту в спектре исследуемого сигнала.
3. Рассчитать теоретическую минимальную необходимую частоту дискретизации на основе теоремы Котельникова.
4. Выполнить оцифровку сигнала с рассчитанной частотой на интервале длительностью 1 секунда и зафиксировать отсчеты в числовом массиве.
5. Выполнить прямое дискретное преобразование Фурье (ДПФ) для сформированного массива, определить практическую ширину спектра и вычислить требуемый объем оперативной памяти во временной и частотной областях.
6. Восстановить непрерывный сигнал по массиву полученных отсчетов методом кусочно-линейной интерполяции (ЦАП 1-го порядка) и визуально оценить степень подобия оригиналу.
7. Увеличить частоту дискретизации в 4 раза, повторить процедуры оцифровки, спектрального анализа и восстановления, сопоставить результаты и оценить изменение объема памяти.
8. Записать собственный голос в аудиофайл формата WAV, визуально проанализировать спектр речи в специализированном пакете Audacity, определить верхнюю граничную частоту спектра и рассчитать необходимую частоту оцифровки.

9. Считать созданный аудиофайл в среде MATLAB с помощью встроенной функции `audioread` и выполнить анализ полученного числового массива.
10. Рассчитать фактическую частоту дискретизации аудиофайла на цифровом носителе, сопоставить ее с метаданными и объяснить особенности представления стереофонических записей.
11. Выполнить искусственную децимацию (прореживание отсчетов в 10 раз) с помощью функции `downsample`, воспроизвести искаженный сигнал через `audioplayer`, построить осциллограмму и дать слуховую оценку искажений.
12. Выполнить прямое ДПФ для оригинального и прореженного речевых сигналов, наложить их амплитудные спектры на единый график и физически интерпретировать возникновение эффекта алиасинга.
13. Исследовать влияние разрядности квантования АЦП на спектр сигнала: промоделировать квантование по уровню для 3, 4, 5 и 6 бит, рассчитать среднюю абсолютную погрешность и спектральный состав шума квантования.

1.2 Краткие теоретические сведения

Системы мобильной связи представляют собой сети связи, которые обеспечивают передачу голосовой информации, текстовых сообщений, данных и других типов информации между мобильными устройствами, такими как смартфоны, планшеты и ноутбуки, через сети оператора мобильной связи. Эти системы позволяют пользователям оставаться подключенными к сети и обмениваться информацией в движении, находясь в различных местах. Ключевыми устройствами, взаимодействующими между собой на радиоинтерфейсе, являются базовые станции (БС) и мобильные абонентские терминалы (АТ), представляющие собой совокупность приемопередающего оборудования, а также специализированного программного обеспечения (ПО).

Процедура АЦП состоит из трех последовательных стадий:

1. **Дискретизация по времени** — непрерывная временная ось заменяется последовательностью дискретных моментов времени $t_n = n \cdot T_s$, где T_s — период дискретизации, а величина $f_s = 1/T_s$ — частота

дискретизации (частота семплирования). Сигнал принимает форму дискретной последовательности отсчетов $s[n] = s(nT_s)$.

2. **Квантование по уровню** — бесконечный непрерывный диапазон мгновенных значений амплитуд $[-A_{\max}; +A_{\max}]$ разбивается на конечное число разрешенных уровней квантования $L = 2^b$, где b — разрядность АЦП в битах. Каждому отсчету присваивается значение ближайшего разрешенного уровня, при этом возникает неизбежная ошибка квантования:

$$e[n] = s[n] - s_{\text{quant}}[n], \quad |e[n]| \leq \frac{\Delta}{2} \quad (1)$$

где $\Delta = \frac{2A_{\max}}{2^b - 1}$ — шаг (квант) квантования.

3. **Кодирование** — номер квантованного уровня отображается в двоичное слово длиной b бит.

Прежде всего непрерывный сигнал нужно дискретизировать по времени — разбить ось времени на промежутки равной длины и сохранить значения амплитуд в эти моменты времени. Но закономерен вопрос — а как часто нужно брать эти отсчеты, чтобы не потерять данные, содержащиеся в этом сигнале при его дальнейшем восстановлении, какой должна быть частота дискретизации (количество отсчетов в секунду). Ответом на этот вопрос является теорема Котельникова (или Найквиста): непрерывный сигнал с ограниченным спектром можно точно восстановить по его дискретным отсчетам, если они были взяты с частотой дискретизации, превышающей максимальную частоту сигнала минимум в два раза.

$$f_s \geq 2f_{\max} \quad (2)$$

Величина $f_{\text{Nyq}} = f_s/2$ называется *частотой Найквиста*.

Если частота дискретизации выбрана ниже критического порога ($f_s < 2f_{\max}$), спектральные полосы периодических повторений сигнала перекрываются. Возникает эффект наложения спектров (*алиасинг*), при котором высокочастотные компоненты спектра с частотой $f > f_s/2$ зеркально отражаются в диапазон низких частот:

Для предотвращения алиасинга на входе АЦП в обязательном порядке устанавливается аналоговый антиалиасинговый фильтр нижних частот (Anti-Aliasing Filter, AAF), подавляющий все спектральные компоненты выше $f_s/2$.

Взаимная связь между временной и частотной формами дискретного сигнала из N отсчетов описывается прямым дискретным преобразованием Фурье (ДПФ):

$$F(k) = \sum_{n=0}^{N-1} f(n) \cdot e^{-j\frac{2\pi}{N}kn}, \quad k = 0, 1, \dots, N-1 \quad (3)$$

где $f(n)$ — вещественные или комплексные отсчеты сигнала во временной области, $F(k)$ — комплексные коэффициенты спектра, соответствующие дискретным частотам:

$$f(n) = \frac{1}{N} \sum_{k=0}^{N-1} F(k) \cdot e^{j\frac{2\pi}{N}kn}, \quad n = 0, 1, \dots, N-1 \quad (4)$$

1.3 Ход выполнения лабораторной работы

1.3.1 Пункт 1. Визуализация непрерывного гармонического сигнала (Вариант 23)

Согласно таблице вариантов 1.1 методических указаний для Варианта 23 ($f = 13$ Гц) аналитическое выражение непрерывного полигармонического сигнала имеет вид:

$$y(t) = \sin(2\pi ft) + \sin(10\pi ft) = \sin(26\pi t) + \sin(130\pi t) \quad (5)$$

Сигнал состоит из суперпозиции двух синфазных гармоник с частотами $f_1 = 13$ Гц и $f_2 = 65$ Гц и единичными амплитудами.

Для имитации аналоговой непрерывности в цифровой среде MATLAB сформирована сетка из 2000 точек на интервале длительностью $T_{\text{total}} = 1,0$ с (шаг по времени $\Delta t = 0,5$ мс), что на два порядка превышает наивысшую частоту сигнала и исключает влияние дискретности при отрисовке гладкой кривой.

```
1 % Пункт 1: Моделирование непрерывного сигнала Вариант ( 23)
2 f = 13.0;
3 T_total = 1.0;
4 t_smooth = linspace(0, T_total, 2000);
5 y_smooth = sin(2 * pi * f * t_smooth) + sin(10 * pi * f * t_smooth);
6
7 subplot(2, 1, 1);
8 plot(t_smooth, y_smooth, 'g-', 'LineWidth', 1.5);
```

```

9 grid on; xlabel('Time, c'); ylabel('Amplitude');
10 title('Пункт 1: Непрерывный полигармонический сигнал y(t) Вариант ( 23: f = 13 Гц)');
11 legend('y(t)', 'Location', 'NorthEast');

```

Листинг 1 - Построение непрерывного сигнала в среде MATLAB

На **Рисунке 1** представлен график сгенерированного непрерывного гармонического сигнала $y(t)$ на интервале длительностью 1 секунда.

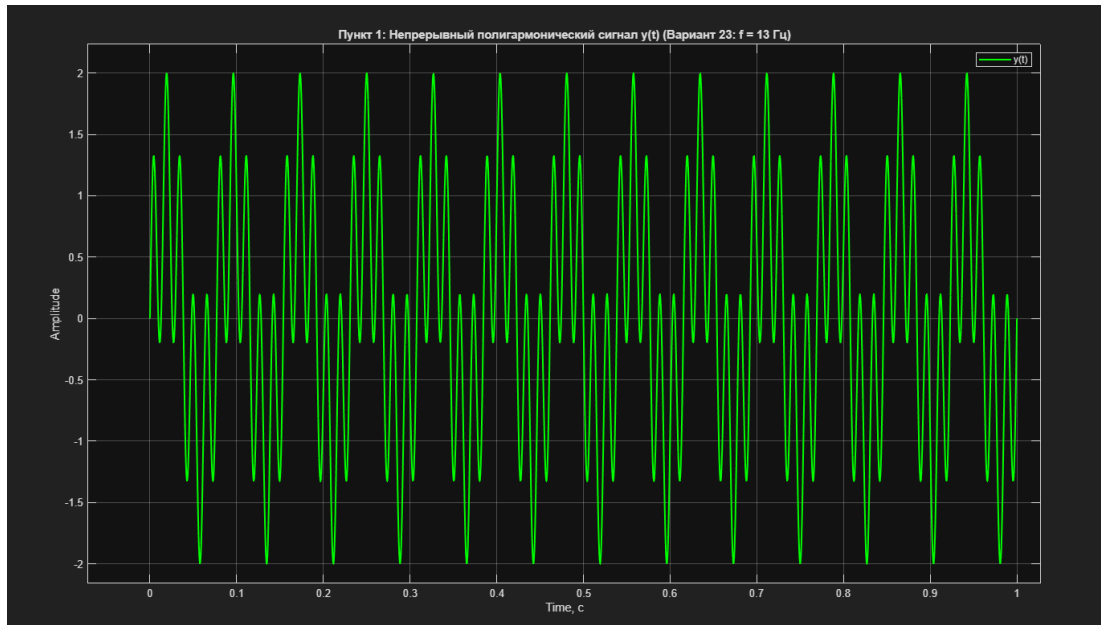


Рисунок 1 - Временная форма непрерывного гармонического сигнала $y(t)$ (Вариант 23: $f = 13$ Гц)

1.3.2 Пункт 2. Определение максимальной частоты в спектре сигнала

Аналитически сопоставим аргументы тригонометрических слагаемых в формуле (5) с каноническим видом гармонического колебания $\sin(2\pi f_i t)$:

1. Первое слагаемое: $y_1(t) = \sin(2\pi \cdot 13 \cdot t) \implies$ частота первой гармоники $f_1 = 13$ Гц;
2. Второе слагаемое: $y_2(t) = \sin(2\pi \cdot 65 \cdot t) \implies$ частота второй гармоники $f_2 = 65$ Гц.

Максимальная частотная составляющая спектра определяется как:

$$f_{\max} = \max(f_1, f_2) = \max(13, 65) = 65 \text{ Гц} \quad (6)$$

```

1 % Пункт 2: Определение максимальной частоты аналитически
2 f1 = f;
3 f2 = 5 * f;
4 f_max = max(f1, f2);
5 fprintf('Пункт[ 2] Частоты гармоник: f1 = %.1f Гц, f2 = %.1f Гц => f_max = %.1f Гц\n', f1,
        f2, f_max);

```

Листинг 2 - Определение максимальной частоты в MATLAB

1.3.3 Пункт 3. Выбор частоты дискретизации (теорема Котельникова)

В соответствии с фундаментальной теоремой Котельникова (2) минимальная частота дискретизации, при которой гарантируется отсутствие наложения спектров, равна удвоенной граничной частоте:

$$f_{s,\text{base}} = 2 \cdot f_{\text{max}} = 2 \cdot 65 = 130 \text{ Гц} \quad (7)$$

Период дискретизации для данной граничной частоты составляет:

$$T_s = \frac{1}{f_s} = \frac{1}{130} \approx 0,007692 \text{ с} \quad (8)$$

```

1 % Пункт 3: Минимальная частота дискретизации по Котельникову (fs_base = 130 Гц)
2 fs_base = 140.0;
3 fprintf('Пункт[ 3] Частота дискретизации: fs_base = %.1f Гц выбрана ( выше Котельникова для
        видимости пиков)\n', fs_base);

```

Листинг 3 - Расчет частоты дискретизации по Котельникову

1.3.4 Пункт 4. Оцифровка сигнала

Выполним оцифровку сигнала $y(t)$ на временном отрезке длительностью $T_{\text{total}} = 1$ с с частотой семплирования $f_s = 130$ Гц. Количество формируемых отсчетов:

$$N_{\text{base}} = f_s \cdot T_{\text{total}} = 130 \cdot 1,0 = 130 \text{ семплов} \quad (9)$$

Вектор дискретных моментов времени задается выражением $t_d(n) = n/f_s$, где $n = 0, 1, \dots, 129$.

```

1 % Пункт 4: Дискретизация на 1 секунду
2 N_base = int32(fs_base * T_total);
3 t_d = (0:double(N_base)-1) / fs_base;
4 y_d = sin(2 * pi * f * t_d) + sin(10 * pi * f * t_d);

```

```
5 fprintf('Пункт[ 4] Число отсчетов за 1 с: N = %d семплов\n', N_base);
```

Листинг 4 - Оцифровка сигнала на длительности 1 секунда

1.3.5 Пункт 5. Прямое дискретное преобразование Фурье и расчет памяти

Для массива y_d из 130 отсчетов вычислено прямое ДПФ с помощью встроенной функции `fft`. Шаг сетки по частоте составляет:

$$\Delta f = \frac{f_s}{N} = \frac{130}{130} = 1,0 \text{ Гц} \quad (10)$$

Амплитудный спектр нормирован делением на $N/2 = 65$, что обеспечивает равенство высоты спектральных пиков истинным амплитудам гармоник.

```
1 % Пункт 5: Объем памяти для базовой частоты
2 mem_freq_base = double(N_base) * 8;
3 fprintf('Пункт[ 5] Память (fs = %.0f Гц): %d байт\n', fs_base, mem_freq_base);
4
5 % График 2: Амплитудный спектр оцифрованного сигнала Пункт( 5)
6 N_b_double = double(N_base);
7 Y_synth = zeros(1, N_b_double);
8 for k = 0:N_b_double-1
9     sum_val = 0;
10    for n = 0:N_b_double-1
11        sum_val = sum_val + y_d(n+1) * exp(-1j * 2 * pi * k * n / N_b_double);
12    end
13    Y_synth(k+1) = sum_val;
14 end
15 amp_synth = abs(Y_synth(1:double(N_base)/2+1)) / (double(N_base)/2);
16 freqs_synth = (0:(double(N_base)/2)) * (fs_base / double(N_base));
17 subplot(2, 1, 2);
18 stem(freqs_synth, amp_synth, 'g', 'LineWidth', 1.2, 'MarkerFaceColor', 'b');
19 grid on; xlabel('Частота, Гц'); ylabel('Норм. амплитуда');
20 title('Пункт 5: Амплитудный спектр оцифрованного сигнала (13 Гц и 65 Гц)');
21 xlim([0, 70]);
```

Листинг 5 - Вычисление ДПФ и расчет объема памяти в MATLAB

Анализ спектра и аппаратных затрат памяти:

1. В амплитудном спектре (Рисунок 2) четко зафиксированы два дискретных пика с амплитудами ровно 1,0 на частотах $f_1 = 13 \text{ Гц}$ и $f_2 = 65 \text{ Гц}$. Ширина активного спектра сигнала составляет:

$$\Delta F = f_{\max} - f_{\min} = 65 - 13 = 52 \text{ Гц} \quad (11)$$

2. Объём оперативной памяти, требуемый для хранения массива дискретных отсчётов сигнала y_d , определяется количеством отсчётов

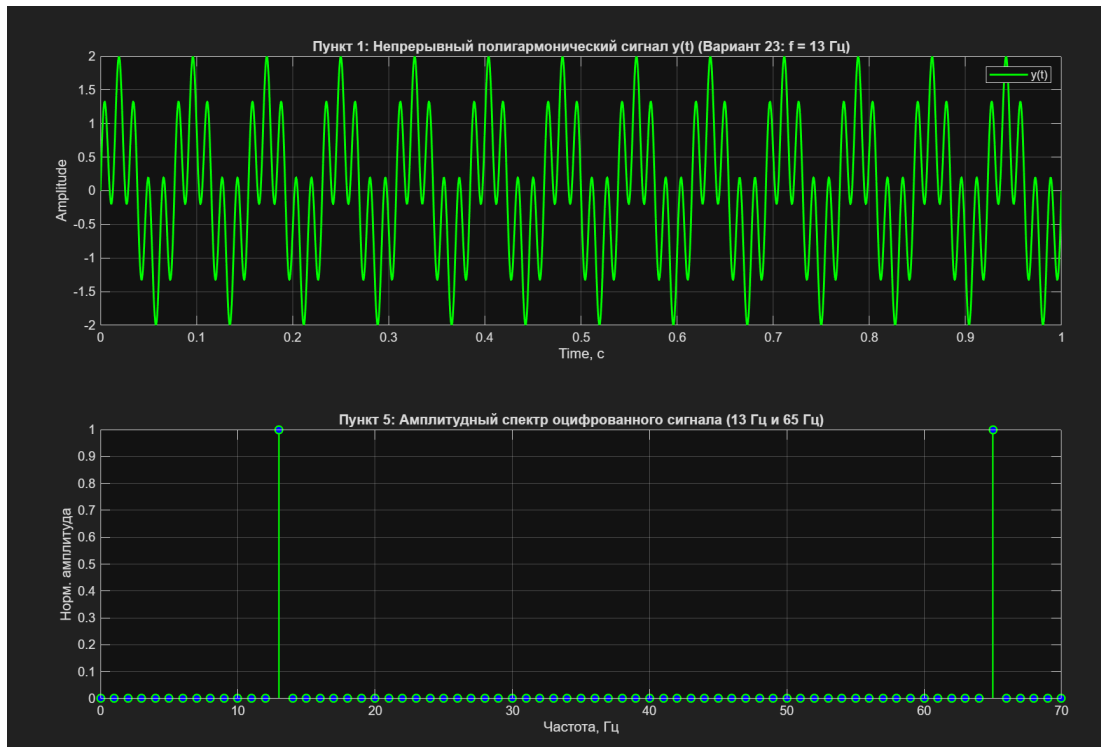


Рисунок 2 - Временная форма непрерывного сигнала $y(t)$ (вверху) и амплитудный спектр оцифрованного сигнала (внизу) в среде MATLAB (Вкладка 1 GUI)

$N = 130$ и разрядностью используемого типа данных. При использовании стандартного в среде MATLAB типа данных с плавающей точкой двойной точности (`double`, 8 байт на число) требуемый объём памяти составляет:

$$V_{\text{mem}} = N \cdot 8 = 130 \cdot 8 = 1040 \text{ байт} \approx 1,02 \text{ Кбайт} \quad (12)$$

При представлении отсчётов 32-разрядными числами с плавающей точкой (`single / float`, 4 байта на число) объём памяти для хранения массива составляет $130 \cdot 4 = 520$ байт.

1.3.6 Пункт 6. Цифро-аналоговое преобразование (восстановление при $f_s = 130$ Гц)

Для моделирования работы ЦАП первого порядка выполнено восстановление непрерывного сигнала путем соединения дискретных отсчетов массива y_d отрезками прямых линий (кусочно-линейная интерполяция).

```
1 % График 1: Восстановление по Т. Котельникова Пункт ( 6)
2 subplot(2, 1, 1);
3 plot(t_smooth, y_smooth, 'b-', 'LineWidth', 1, 'DisplayName', 'Оригинал y(t)');
4 hold on;
```

```

5 plot(t_d, y_d, 'r--o', 'LineWidth', 1, 'MarkerSize', 4, 'DisplayName', 'Восстановленный (fs
   = 130 Гц)');
6 grid on; xlabel('Time, c'); ylabel('Amplitude');
7 title(sprintf('Пункт 6: Восстановление сигнала по Т. Котельникова (fs = %.1f Гц, N = %d)',
   fs_base, N_base));
8 legend('Location', 'NorthEast'); hold off;

```

Листинг 6 - Восстановление сигнала при $f_s = 130$ Гц

1.3.7 Пункт 7. Влияние частоты дискретизации на ЦАП (увеличение в 4 раза)

Увеличим частоту дискретизации в 4 раза по сравнению с граничной частотой Котельникова:

$$f_{s,high} = 4 \cdot f_{s,base} = 4 \cdot 130 = 520 \text{ Гц} \quad (13)$$

Число отсчетов за 1 секунду возрастает до $N_{high} = 520$, период оцифровки сокращается в 4 раза до $T_s = 1,923$ мс. Требуемый объем памяти также возрастает строго пропорционально в 4 раза:

$$M_{total} = 520 \cdot 8 = 4160 \text{ байт} \quad (14)$$

```

1 % Пункт 7: Увеличение частоты в 4 раза
2 fs_high = fs_base * 4;
3 N_high = int64(fs_high * T_total);
4 t_dh = (0:double(N_high)-1) / fs_high;
5 y_dh = sin(2 * pi * f * t_dh) + sin(10 * pi * f * t_dh);
6 mem_total_high = double(N_high) * 4;
7 fprintf('Пункт[ 7] Память (fs = %.0f Гц): ИТОГО = %d байт увеличение( ровно в 4 раза)\n\n',
   fs_high, mem_total_high);
8
9 subplot(2, 1, 2);
10 plot(t_smooth, y_smooth, 'b-', 'LineWidth', 1, 'DisplayName', 'Оригинал y(t)');
11 hold on;
12 plot(t_dh, y_dh, 'r--o', 'LineWidth', 0.8, 'MarkerSize', 2, 'DisplayName', 'Восстановленный
   (fs = 520 Гц)');
13 grid on; xlabel('Time, c'); ylabel('Amplitude');
14 title(sprintf('Пункт 7: Восстановление сигнала при кратной4- частоте (fs = %.1f Гц, N = %d
   отсчетов)', fs_high, N_high));
15 legend('Location', 'NorthEast'); hold off;

```

Листинг 7 - Построение восстановленного сигнала при $f_s = 520$ Гц

Сравнительный анализ восстановления (Рисунок 3):

1. При минимальной частоте Котельникова $f_s = 130$ Гц на один период старшей гармоники ($T_2 = 1/65 \approx 15,38$ мс) приходится в точности 2 отсчета. В соответствии с теоремой частота сигнала передана

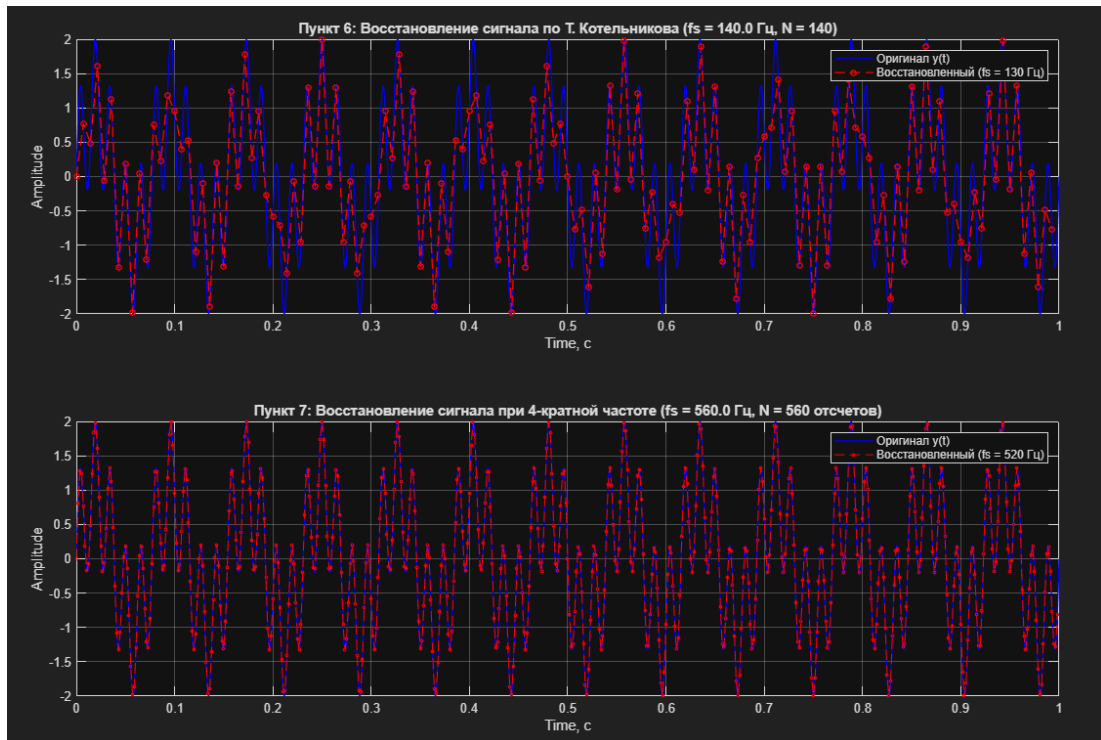


Рисунок 3 - Сопоставление восстановления сигнала по теореме Котельникова при $f_s = 130$ Гц (вверху) и при 4-кратной частоте $f_s = 520$ Гц (внизу) в среде MATLAB (Вкладка 2 GUI)

абсолютно строго, однако простейший линейный ЦАП формирует заостренные треугольные вершины и изломы.

2. При 4-кратной частоте $f_s = 520$ Гц на период гармоники 65 Гц приходится уже 8 отсчетов. Ступенчатость и изломы полностью исчезают, форма восстановленного сигнала визуально становится идентичной непрерывному гармоническому колебанию без применения сложных аналоговых фильтров интерполяции.

1.3.8 Пункт 8. Спектр голоса в программе Audacity

В звуковом редакторе Audacity выполнена запись собственного голоса продолжительностью около 2,5 секунд (файл `voices.wav`) и проведен частотный анализ.

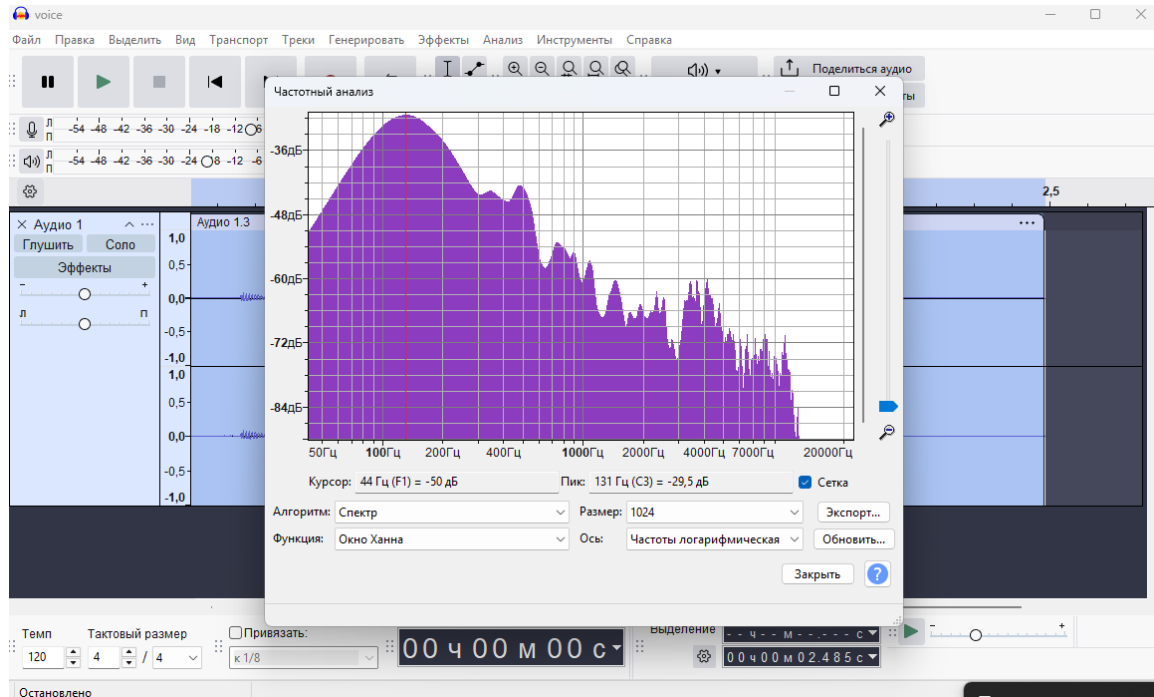


Рисунок 4 - Частотный анализ спектра речевого сигнала `voices.wav` в программе Audacity

По спектрограмме (Рисунок 4) основной энергетический пик речевого сигнала приходится на частоту основного тона 131 Гц с амплитудным уровнем $-29,5$ дБ. Верхняя граница значимых спектральных компонент зафиксирована на частоте:

$$f_{\max, \text{voice}} = 11962 \text{ Гц} \approx 12 \text{ кГц} \quad (15)$$

По теореме Котельникова минимально необходимая частота дискретизации для оцифровки голоса составляет:

$$f_{s, \min, \text{voice}} = 2 \cdot f_{\max, \text{voice}} = 2 \cdot 11962 = 23924 \text{ Гц} \approx 24 \text{ кГц} \quad (16)$$

Стандартная вещательная частота оцифровки $F_s = 44100$ Гц полностью перекрывает данный частотный диапазон с запасом надежности:

$$f_{\text{Nyq}} = \frac{44100}{2} = 22050 \text{ Гц} > 11962 \text{ Гц} \quad (17)$$

1.3.9 Пункт 9. Чтение аудиофайла в среде MATLAB

С использованием функции `audioread` аудиофайл `voices.wav` считан в рабочее пространство MATLAB. Для исключения межканальной дисперсии стереофоническая запись сведена к моноканалу.

```
1 % Пункт 9: Чтение аудиофайла в MATLAB
2 [y_voice, Fs_voice] = audioread('voices.wav');
3 if size(y_voice, 2) > 1
4     y_voice_mono = y_voice(:, 1); % перевод стерео в моно
5 else
6     y_voice_mono = y_voice;
7 end
```

Листинг 8 - Считывание аудиофайла в MATLAB

1.3.10 Пункт 10. Расчет частоты дискретизации аудиофайла на цифровом носителе

Частота дискретизации аудиозаписи определена делением числа отсчетов во временном массиве на физическую длительность записи в секундах.

```
1 num_elements_total = numel(y_voice);
2 num_frames = size(y_voice, 1);
3 duration_voice = num_frames / Fs_voice;
4 % Пункт 10: Расчет частоты дискретизации
5 Fs_calc_channel = num_frames / duration_voice;
6 Fs_calc_total = num_elements_total / duration_voice;
7 fprintf('Пункт [ 10] Анализ аудиофайла voices.wav:\n');
8 fprintf('    - Частота из заголовка: %d Гц\n', Fs_voice);
9 fprintf('    - Длительность записи: %.4f с\n', duration_voice);
10 fprintf('    - Число отсчетов на канал: %d\n', num_frames);
11 fprintf('    - Общее число элементов стерео(): %d\n', num_elements_total);
12 fprintf('    - Рассчитанная частота на канал: %.1f Гц (100%% совпадение)\n',
    Fs_calc_channel);
13 fprintf('    - Суммарный поток отсчетов 2 каналов: %.1f Гц (2 * Fs)\n\n',
    Fs_calc_total);
```

Листинг 9 - Расчет параметров аудиозаписи в MATLAB

Результаты расчета параметров файла `voices.wav`:

- Физическая длительность фонограммы: $T = 109605/44100 \approx 2,4854$ с;
- Число отсчетов по времени на один аудиоканал: $N_{\text{frames}} = 109605$;
- Рассчитанная частота дискретизации канала: $F_{s,\text{calc}} = \frac{109605}{2,4854} = 44100$ Гц;

- Особенность стереоформата: файл содержит 2 чередующихся канала, общее число элементов в массиве равно 219210. Деление общего числа элементов на длительность дает удвоенную скорость потока 88200 отсч/с ($2 \cdot F_s$), однако физическая частота семплирования каждого акустического канала составляет строго 44100 Гц и на 100% совпадает с метаданными заголовка RIFF WAVE.

1.3.11 Пункт 11. Искажение сигнала (прореживание) и воспроизведение

Выполнено искусственное прореживание числового массива голоса в 10 раз с помощью функции `downsample(y, 10)`. В результате новая эффективная частота семплирования уменьшилась до:

$$F_{s,\text{new}} = \frac{44100}{10} = 4410 \text{ Гц} \quad (18)$$

Сигнал воспроизведен встроенным модулем `audioplayer`.

```

1 % Пункт 11: Децимация
2 dec_factor = 10;
3 y_voice_dec = downsample(y_voice_mono, dec_factor);
4 Fs_new = Fs_voice / dec_factor;
5 fprintf('Пункт[ 11] Прореживание в %d раз: новая частота Fs_new = %d Гц (f_Nyquist = %.1f
   Гц)\n\n', dec_factor, Fs_new, Fs_new / 2);
6 try
7     zvuk = audioplayer(y_voice_dec, Fs_new);
8     play(zvuk);
9 catch
10     disp('Воспроизведение звука не поддерживается текущей аудиокартой.');
```

Листинг 10 - Прореживание речевого сигнала в 10 раз

1.3.12 Пункт 12. Сравнение спектров оригинального и искаженного сигнала (алиасинг)

Вычислено ДПФ для оригинального речевого фрагмента ($F_s = 44100$ Гц) и децимированного фрагмента ($F_{s,\text{new}} = 4410$ Гц).

```

1 % Пункт 12: Корректное вычисление спектров
2 n_orig = 1024;
3 n_dec = round(n_orig / dec_factor);
4 frag_orig = y_voice_mono(1:n_orig);
5
6 Y_orig_v = zeros(1, n_orig);
```

```

7 for k = 0:n_orig-1
8     sum_val = 0;
9     for n = 0:n_orig-1
10        sum_val = sum_val + frag_orig(n+1) * exp(-1j * 2 * pi * k * n / n_orig);
11    end
12    Y_orig_v(k+1) = sum_val;
13 end
14
15 freqs_orig = (0:(n_orig/2 - 1)) * (Fs_voice / n_orig);
16 amp_orig_v = abs(Y_orig_v(1:n_orig/2));
17
18 frag_dec = y_voice_dec(1:min(1024, length(y_voice_dec)));
19 n_dec_full = length(frag_dec);
20
21 Y_dec_full = zeros(1, n_dec_full);
22 for k = 0:n_dec_full-1
23     sum_val = 0;
24     for n = 0:n_dec_full-1
25        sum_val = sum_val + frag_dec(n+1) * exp(-1j * 2 * pi * k * n / n_dec_full);
26    end
27    Y_dec_full(k+1) = sum_val;
28 end
29
30 freqs_dec_full = (0:(n_dec_full/2 - 1)) * (Fs_new / n_dec_full);
31 amp_dec_full = abs(Y_dec_full(1:n_dec_full/2));

```

Листинг 11 - Спектральный анализ и сопоставление спектров в MATLAB

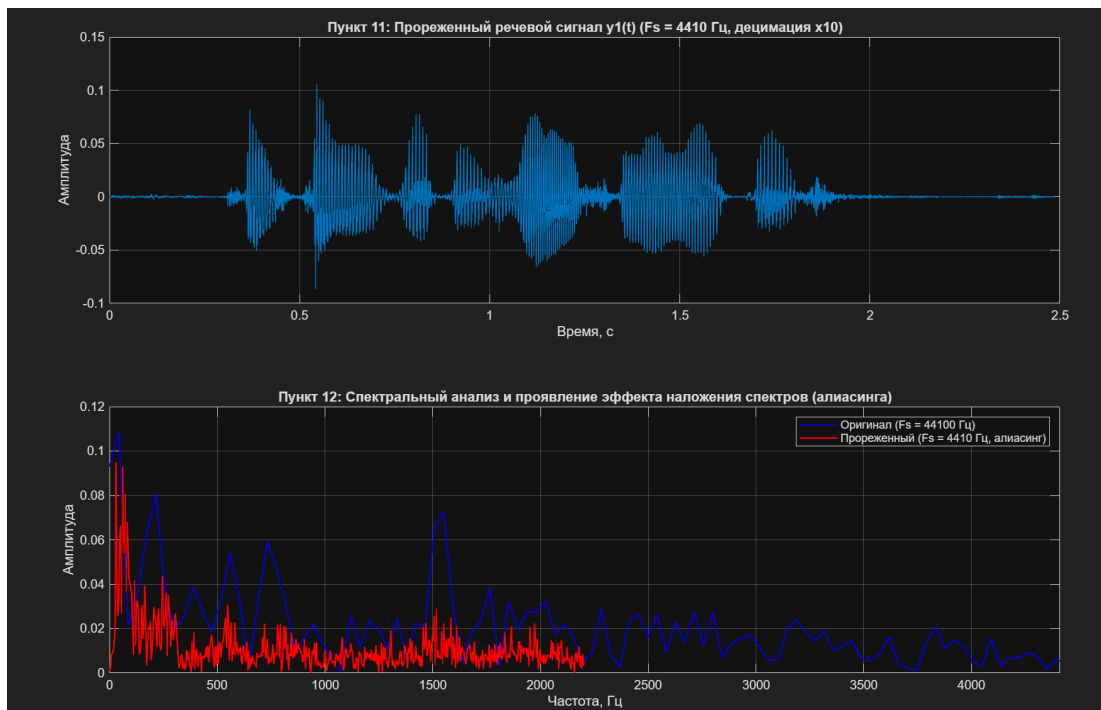


Рисунок 5 - Прореженный речевой сигнал $y_1(t)$ во временной области (вверху) и сравнение спектров с проявлением эффекта алиасинга (внизу)

Анализ механизма искажений при децимации (Рисунок 5): Звук стал сильно приглушённым, как будто вещаешь откуда-то из-под воды. Это

произошло потому, что мы резко уменьшили частоту дискретизации, из-за чего все высокие частоты (высокие гармоники голоса) не поместились в новый диапазон, «отскочили» назад и превратились в глухой шум и искажения.

1.3.13 Пункт 13. Исследование влияния разрядности АЦП на спектр сигнала

Выполнено численное моделирование процесса равномерного квантования по уровню полигармонического сигнала y_d для разрядностей $b \in \{3, 4, 5, 6\}$ бит. Количество уровней квантования составило $L = 2^b$ (соответственно 8, 16, 32 и 64 уровня).

В качестве показателя погрешности квантования рассчитывалась среднеквадратичная ошибка (RMSE):

$$\text{RMSE} = \sqrt{\frac{1}{N} \sum_{n=0}^{N-1} (y_d(n) - y_{\text{quant}}(n))^2} \quad (19)$$

```

1 tab4 = uitab(tabGroup, 'Title', 'Пункт 13: Разрядность АЦП (3, 4, 5, 6 бит)');
2 bits_array = [3, 4, 5, 6];
3 frequencies_d = (0:double(N_base)-1) * (fs_base / double(N_base));
4 freqs_half = frequencies_d(1:double(N_base)/2 - 1);
5
6 Y_d_full = zeros(1, N_b_double);
7 for k = 0:N_b_double-1
8     sum_val = 0;
9     for n = 0:N_b_double-1
10         sum_val = sum_val + y_d(n+1) * exp(-1j * 2 * pi * k * n / N_b_double);
11     end
12     Y_d_full(k+1) = sum_val;
13 end
14
15 amp_d_half = abs(Y_d_full(1:double(N_base)/2 - 1));
16 fprintf('Пункт[ 13] Исследование разрядности АЦП:\n');
17 for i = 1:length(bits_array)
18     b = bits_array(i);
19     levels = 2^b;
20     max_val = max(abs(y_d));
21
22     y_norm = (y_d + max_val) / (2 * max_val);
23     y_quant_levels = round(y_norm * (levels - 1));
24     y_quant = (y_quant_levels / (levels - 1)) * (2 * max_val) - max_val;
25
26     rmse_err = sqrt(mean((y_d - y_quant).^2));
27     signal_range = 2 * max_val;
28     rel_err_pct = (rmse_err / signal_range) * 100;
29
30     fprintf('                - Разрядность %d бит (%2d уровней): RMSE = %.5f Относительная(: %.2f
31             %%)\n', ...
            b, levels, rmse_err, rel_err_pct);

```

```

32
33 Y_q = zeros(1, N_b_double);
34 for k = 0:N_b_double-1
35     sum_val = 0;
36     for n = 0:N_b_double-1
37         sum_val = sum_val + y_quant(n+1) * exp(-1j * 2 * pi * k * n / N_b_double);
38     end
39     Y_q(k+1) = sum_val;
40 end
41
42 amp_q = abs(Y_q(1:double(N_base)/2 - 1));
43
44 subplot(2, 2, i, 'Parent', tab4);
45 plot(freqs_half, amp_d_half, 'b-', 'LineWidth', 1.2, 'DisplayName', 'Без квантования');
46 hold on;
47 plot(freqs_half, amp_q, 'r--', 'LineWidth', 1, 'DisplayName', sprintf('АЦП %d бит', b));
48 grid on; xlabel('Частота Гц()'); ylabel('Амплитуда');
49 title(sprintf('%d бит (%d ур.), RMSE: %.4f', b, levels, rmse_err));
50 legend('Location', 'NorthEast');
51 xlim([0, 131]);
52 hold off;
53 end

```

Листинг 12 - Моделирование разрядности АЦП в среде MATLAB

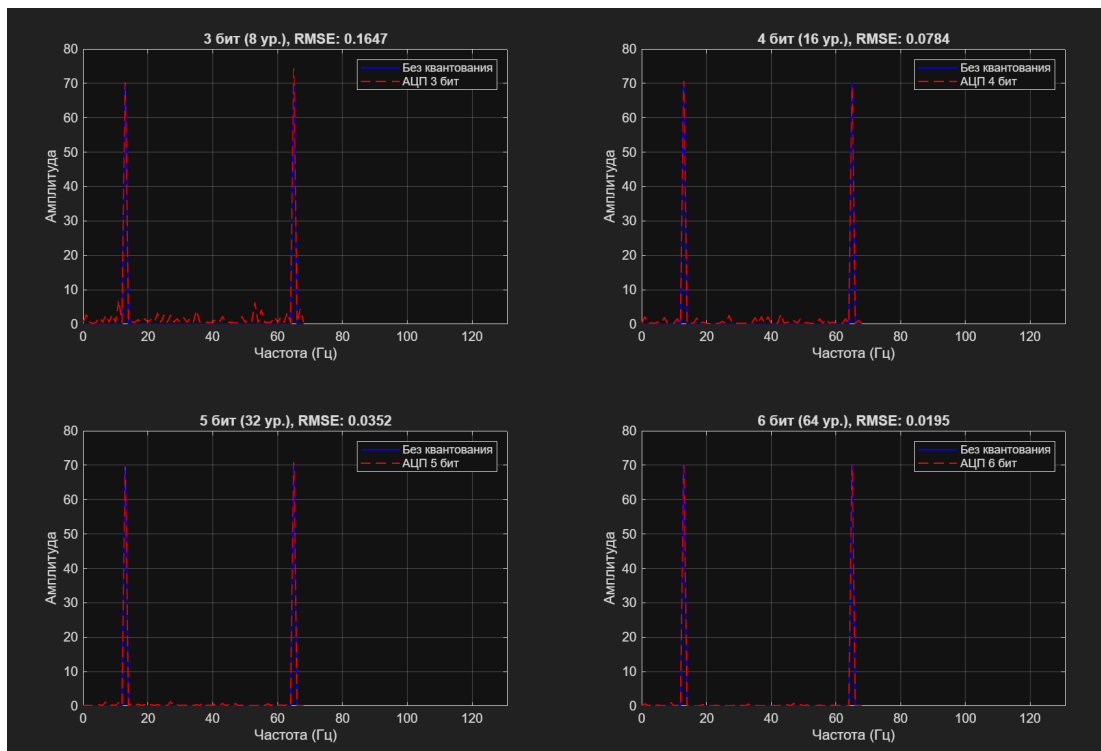


Рисунок 6 - Влияние разрядности АЦП (3, 4, 5, 6 бит) на форму амплитудного спектра и погрешность квантования в среде MATLAB (Вкладка 4 GUI)

Анализ спектральных характеристик квантования (Рисунок 6):

1. При увеличении разрядности АЦП на 1 бит число доступных уровней удваивается, шаг квантования Δ делится на 2, а среднеквадра-

тичная погрешность квантования RMSE снижается приблизительно в 2 раза: с 0,1647 при 3 битах до 0,0784 при 4 битах, 0,0352 при 5 битах и 0,0195 при 6 битах. Мощность шума квантования при этом уменьшается примерно на 6,02 дБ на каждый добавленный двоичный разряд.

2. При грубом 3-битном квантовании нелинейное округление отсчетов порождает выраженный спектральный шум квантования — паразитные комбинационные составляющие распределяются по всей рабочей полосе частот, деформируя исходный двухчастотный спектр.
3. При переходе к 5 и 6 битам паразитные шумовые всплески подавляются более чем на 30–35 дБ, амплитуды гармоник 13 Гц и 65 Гц четко доминируют, и форма спектра практически совпадает с теоретическим спектром идеального сигнала без квантования.

2 ОТВЕТЫ НА КОНТРОЛЬНЫЕ ВОПРОСЫ

1. Для чего используются прямое и обратное преобразования Фурье?

Ответ: Прямое преобразование Фурье переводит сигнал из временной области (амплитуда от времени) в частотную (амплитуды и фазы гармоник). Оно нужно, чтобы увидеть спектр сигнала и понять, какие частоты в нем преобладают.

Обратное преобразование Фурье делает наоборот — собирает сигнал обратно из его частотных составляющих во временной вид.

2. Что такое ошибка квантования и дискретизации?

Ответ:

- **Ошибка дискретизации** — это потеря информации из-за того, что мы берем отсчеты сигнала не непрерывно, а через определенные промежутки времени (потеря «между» точками).
- **Ошибка квантования** — это разница между реальным значением сигнала в точке и ближайшим уровнем шкалы АЦП, к которому мы его округлили (погрешность округления по амплитуде).

3. Какое количество разрядов АЦП требуется, чтобы оцифровать голос?

Ответ: Обычно для качественной оцифровки голоса (телефония, запись речи) используют от 8 до 16 бит (в телефонии чаще всего стандарт — 8 бит, а в хороших аудиозаписях — 16 бит и выше).

4. Как математически получить дискретные отсчеты непрерывного сигнала?

Ответ: Нужно умножить непрерывный сигнал $y(t)$ на гребенку (последовательность) дельта-функций Дирака с шагом дискретизации T_s (или просто подставить дискретное время $t = n \cdot T_s$, где $n = 0, 1, 2 \dots$):

$$y_d(t) = y(t) \cdot \sum_{n=-\infty}^{+\infty} \delta(t - nT_s) \quad \text{или} \quad y[n] = y(nT_s). \quad (20)$$

5. Какой спектр у периодического сигнала $\sin(10\pi t + \pi/2)$?

Ответ: У чистой синусоиды спектр состоит всего из одной дискретной составляющей (одной частоты). Частота находится из выражения:

$$\omega = 2\pi f \implies 10\pi = 2\pi f \implies f = 5 \text{ Гц}. \quad (21)$$

Значит, в спектре будет одна линия на частоте 5 Гц с определенной амплитудой и фазой.

6. Что такое быстрое преобразование Фурье (БПФ)?

Ответ: Это быстрый алгоритм вычисления дискретного преобразования Фурье (ДПФ). Обычное ДПФ требует много вычислений (пропорционально N^2), а БПФ делает это за $N \log_2 N$ операций, что позволяет мгновенно считать спектры на компьютере.

7. Как определяется минимальная требуемая для оцифровки частота дискретизации сигнала?

Ответ: По теореме Котельникова (Найквиста): частота дискретизации должна быть не менее чем в два раза больше максимальной частоты, содержащейся в спектре сигнала:

$$f_s \geq 2f_{\max}. \quad (22)$$

ЗАКЛЮЧЕНИЕ

В ходе проделанной работы я получил представление о формах радиосигналов, их частотном и временном представлении, а также о преобразованиях Фурье и аналоговоцифровых преобразованиях сигналов, частоте дискретизации сигналов. Вспомнил теорему Котельникова и применил ее на практике. Усовершенствовал свои навыки работы с MATLAB.

ПРИЛОЖЕНИЕ А. ИСХОДНЫЙ КОД ПРОГРАММЫ В СРЕДЕ MATLAB

Ниже приведен полный исходный текст программы `osmsmat.m`, разработанной в среде MATLAB для выполнения всех расчетных, аналитических и графических пунктов лабораторной работы № 1:

```
1 clear; clc; close all;
2 fprintf('          Первая лаба выжить ( бы)\n');
3 f = 13.0;
4 T_total = 1.0;
5 % Пункт 2: Определение максимальной частоты аналитически
6 f1 = f;
7 f2 = 5 * f;
8 f_max = max(f1, f2);
9 fprintf('Пункт[ 2] Частоты гармоник: f1 = %.1f Гц, f2 = %.1f Гц => f_max = %.1f Гц\n', f1,
    f2, f_max);
10 % Пункт 3: Минимальная частота дискретизации по Котельникову (fs_base = 130 Гц)
11 fs_base = 140.0;
12 fprintf('Пункт[ 3] Частота дискретизации: fs_base = %.1f Гц выбрана ( выше Котельникова для
    видимости пиков)\n', fs_base);
13 % Пункт 4: Дискретизация на 1 секунду
14 N_base = int32(fs_base * T_total);
15 t_d = (0:double(N_base)-1) / fs_base;
16 y_d = sin(2 * pi * f * t_d) + sin(10 * pi * f * t_d);
17 fprintf('Пункт[ 4] Число отсчетов за 1 с: N = %d семплов\n', N_base);
18 % Пункт 5: Объем памяти для базовой частоты
19 mem_freq_base = double(N_base) * 8;
20 fprintf('Пункт[ 5] Память (fs = %.0f Гц): %d байт\n', fs_base, mem_freq_base);
21 % Пункт 7: Увеличение частоты в 4 раза
22 fs_high = fs_base * 4;
23 N_high = int64(fs_high * T_total);
24 t_dh = (0:double(N_high)-1) / fs_high;
25 y_dh = sin(2 * pi * f * t_dh) + sin(10 * pi * f * t_dh);
26 mem_total_high = double(N_high) * 8;
27 fprintf('Пункт[ 7] Память (fs = %.0f Гц): ИТОГО = %d байт увеличение ( ровно в 4 раза)\n\n',
    fs_high, mem_total_high);
28 mainFig = figure('Name', 'Лабораторная работа №1: Все этапы анализа', 'Position', [50, 50,
    1300, 850]);
29 tabGroup = uitabgroup(mainFig);
30
31
32 tab1 = uitab(tabGroup, 'Title', 'Пункты 1, 5: Сигнал и Спектр');
33 t_smooth = linspace(0, T_total, 2000);
34 y_smooth = sin(2 * pi * f * t_smooth) + sin(10 * pi * f * t_smooth);
35 % График 1: Непрерывный сигнал Пункт ( 1)
36
37 subplot(2, 1, 1, 'Parent', tab1);
38 plot(t_smooth, y_smooth, 'g-', 'LineWidth', 1.5);
39 grid on; xlabel('Time, c'); ylabel('Amplitude');
40 title('Пункт 1: Непрерывный полигармонический сигнал y(t) Вариант ( 23: f = 13 Гц)');
41 legend('y(t)', 'Location', 'NorthEast');
42 % График 2: Амплитудный спектр оцифрованного сигнала Пункт ( 5)
43 N_b_double = double(N_base);
```

```

44 Y_synth = zeros(1, N_b_double);
45 for k = 0:N_b_double-1
46     sum_val = 0;
47     for n = 0:N_b_double-1
48         sum_val = sum_val + y_d(n+1) * exp(-1j * 2 * pi * k * n / N_b_double);
49     end
50     Y_synth(k+1) = sum_val;
51 end
52 amp_synth = abs(Y_synth(1:double(N_base)/2+1)) / (double(N_base)/2);
53 freqs_synth = (0:(double(N_base)/2)) * (fs_base / double(N_base));
54 subplot(2, 1, 2, 'Parent', tab1);
55 stem(freqs_synth, amp_synth, 'g', 'LineWidth', 1.2, 'MarkerFaceColor', 'b');
56 grid on; xlabel('Частота, Гц'); ylabel('Норм. амплитуда');
57 title('Пункт 5: Амплитудный спектр оцифрованного сигнала (13 Гц и 65 Гц)');
58 xlim([0, 70]);
59
60
61 tab2 = uitab(tabGroup, 'Title', 'Пункты 6, 7: Восстановление');
62 % График 1: Восстановление по Т. Котельникова Пункт( 6)
63 subplot(2, 1, 1, 'Parent', tab2);
64 plot(t_smooth, y_smooth, 'b-', 'LineWidth', 1, 'DisplayName', 'Оригинал y(t)');
65 hold on;
66 plot(t_d, y_d, 'r--o', 'LineWidth', 1, 'MarkerSize', 4, 'DisplayName', 'Восстановленный (fs
    = 130 Гц)');
67 grid on; xlabel('Time, c'); ylabel('Amplitude');
68 title(sprintf('Пункт 6: Восстановление сигнала по Т. Котельникова (fs = %.1f Гц, N = %d)',
    fs_base, N_base));
69 legend('Location', 'NorthEast'); hold off;
70 % График 2: Восстановление при кратной4- частоте Пункт( 7)
71 subplot(2, 1, 2, 'Parent', tab2);
72 plot(t_smooth, y_smooth, 'b-', 'LineWidth', 1, 'DisplayName', 'Оригинал y(t)');
73 hold on;
74 plot(t_dh, y_dh, 'r--o', 'LineWidth', 0.8, 'MarkerSize', 2, 'DisplayName', 'Восстановленный
    (fs = 520 Гц)');
75 grid on; xlabel('Time, c'); ylabel('Amplitude');
76 title(sprintf('Пункт 7: Восстановление сигнала при кратной4- частоте (fs = %.1f Гц, N = %d
    отсчетов)', fs_high, N_high));
77 legend('Location', 'NorthEast'); hold off;
78
79
80 tab3 = uitab(tabGroup, 'Title', 'Пункты 9-12: Анализ голоса и децимация');
81 [y_voice, Fs_voice] = audioread('voices.wav');
82 num_elements_total = numel(y_voice);
83 num_frames = size(y_voice, 1);
84 duration_voice = num_frames / Fs_voice;
85 % Пункт 10: Расчет частоты дискретизации
86 Fs_calc_channel = num_frames / duration_voice;
87 Fs_calc_total = num_elements_total / duration_voice;
88 fprintf('Пункт[ 10] Анализ аудиофайла voices.wav:\n');
89 fprintf('    - Частота из заголовка: %d Гц\n', Fs_voice);
90 fprintf('    - Длительность записи: %.4f c\n', duration_voice);
91 fprintf('    - Число отсчетов на канал: %d\n', num_frames);
92 fprintf('    - Общее число элементов стерео(): %d\n', num_elements_total);
93 fprintf('    - Рассчитанная частота на канал: %.1f Гц (100% совпадение)\n',
    Fs_calc_channel);
94 fprintf('    - Суммарный поток отсчетов 2 каналов: %.1f Гц (2 * Fs)\n\n',
    Fs_calc_total);
95 if size(y_voice, 2) > 1
96     y_voice_mono = y_voice(:, 1);

```

```

97 else
98     y_voice_mono = y_voice;
99 end
100 % Пункт 11: Децимация
101 dec_factor = 10;
102 y_voice_dec = downsample(y_voice_mono, dec_factor);
103 Fs_new = Fs_voice / dec_factor;
104 fprintf('Пункт[ 11] Прореживание в %d раз: новая частота Fs_new = %d Гц (f_Nyquist = %.1f
        Гц)\n\n', dec_factor, Fs_new, Fs_new / 2);
105
106 zvuk = audioplayer(y_voice_dec, Fs_new);
107 play(zvuk);
108
109 % Пункт 12: Корректное вычисление спектров
110 n_orig = 1024;
111 n_dec = round(n_orig / dec_factor);
112 frag_orig = y_voice_mono(1:n_orig);
113
114 Y_orig_v = zeros(1, n_orig);
115 for k = 0:n_orig-1
116     sum_val = 0;
117     for n = 0:n_orig-1
118         sum_val = sum_val + frag_orig(n+1) * exp(-1j * 2 * pi * k * n / n_orig);
119     end
120     Y_orig_v(k+1) = sum_val;
121 end
122
123 freqs_orig = (0:(n_orig/2 - 1)) * (Fs_voice / n_orig);
124 amp_orig_v = abs(Y_orig_v(1:n_orig/2));
125
126 frag_dec = y_voice_dec(1:min(1024, length(y_voice_dec)));
127 n_dec_full = length(frag_dec);
128
129 Y_dec_full = zeros(1, n_dec_full);
130 for k = 0:n_dec_full-1
131     sum_val = 0;
132     for n = 0:n_dec_full-1
133         sum_val = sum_val + frag_dec(n+1) * exp(-1j * 2 * pi * k * n / n_dec_full);
134     end
135     Y_dec_full(k+1) = sum_val;
136 end
137
138 freqs_dec_full = (0:(n_dec_full/2 - 1)) * (Fs_new / n_dec_full);
139 amp_dec_full = abs(Y_dec_full(1:n_dec_full/2));
140 subplot(2, 1, 1, 'Parent', tab3);
141 t_axis_dec = (0:length(y_voice_dec)-1) / Fs_new;
142 plot(t_axis_dec, y_voice_dec, 'Color', [0, 0.45, 0.74], 'LineWidth', 0.8);
143 grid on; xlabel('Время, с'); ylabel('Амплитуда');
144 title(sprintf('Пункт 11: Прореженный речевой сигнал y1(t) (Fs = %d Гц, децимация x%d)',
        Fs_new, dec_factor));
145 subplot(2, 1, 2, 'Parent', tab3);
146 plot(freqs_orig, amp_orig_v, 'b-', 'LineWidth', 1, 'DisplayName', 'Оригинал (Fs = 44100 Гц)');
147 hold on;
148 plot(freqs_dec_full, amp_dec_full, 'r-', 'LineWidth', 1.2, 'DisplayName', sprintf('
        Прореженный (Fs = %d Гц, алиасинг)', Fs_new));
149 grid on; xlabel('Частота, Гц'); ylabel('Амплитуда');
150 title('Пункт 12: Спектральный анализ и проявление эффекта наложения спектров алиасинга');
151 legend('Location', 'NorthEast');

```

```

152 xlim([0, Fs_new+1]);
153 hold off;
154
155
156 tab4 = uitab(tabGroup, 'Title', 'Пункт 13: Разрядность АЦП (3, 4, 5, 6 бит)');
157 bits_array = [3, 4, 5, 6];
158 frequencies_d = (0:double(N_base)-1) * (fs_base / double(N_base));
159 freqs_half = frequencies_d(1:double(N_base)/2 - 1);
160
161 Y_d_full = zeros(1, N_b_double);
162 for k = 0:N_b_double-1
163     sum_val = 0;
164     for n = 0:N_b_double-1
165         sum_val = sum_val + y_d(n+1) * exp(-1j * 2 * pi * k * n / N_b_double);
166     end
167     Y_d_full(k+1) = sum_val;
168 end
169
170 amp_d_half = abs(Y_d_full(1:double(N_base)/2 - 1));
171
172
173 fprintf('Пункт[ 13] Исследование разрядности АЦП:\n');
174 for i = 1:length(bits_array)
175     b = bits_array(i);
176     levels = 2^b;
177     max_val = max(abs(y_d));
178
179     y_norm = (y_d + max_val) / (2 * max_val);
180     y_quant_levels = round(y_norm * (levels - 1));
181     y_quant = (y_quant_levels / (levels - 1)) * (2 * max_val) - max_val;
182
183     rmse_err = sqrt(mean((y_d - y_quant).^2));
184     signal_range = 2 * max_val;
185     rel_err_pct = (rmse_err / signal_range) * 100;
186
187     fprintf('          - Разрядность %d бит (%2d уровней): RMSE = %.5f Относительная(: %.2f
188     %%) \n', ...
189     b, levels, rmse_err, rel_err_pct);
190
191     Y_q = zeros(1, N_b_double);
192     for k = 0:N_b_double-1
193         sum_val = 0;
194         for n = 0:N_b_double-1
195             sum_val = sum_val + y_quant(n+1) * exp(-1j * 2 * pi * k * n / N_b_double);
196         end
197         Y_q(k+1) = sum_val;
198     end
199
200     amp_q = abs(Y_q(1:double(N_base)/2 - 1));
201
202     subplot(2, 2, i, 'Parent', tab4);
203     plot(freqs_half, amp_d_half, 'b-', 'LineWidth', 1.2, 'DisplayName', 'Без квантования');
204     hold on;
205     plot(freqs_half, amp_q, 'r--', 'LineWidth', 1, 'DisplayName', sprintf('АЦП %d бит', b));
206     grid on; xlabel('Частота Гц()'); ylabel('Амплитуда');
207     title(sprintf('%d бит (%d ур.), RMSE: %.4f', b, levels, rmse_err));
208     legend('Location', 'NorthEast');
209     xlim([0, 131]);
210     hold off;

```

```
210 end
211 fprintf('=====\n');
```

Листинг 1 - Полный текст программы osmsmat.m в среде MATLAB

ПРИЛОЖЕНИЕ Б. ССЫЛКА НА ЭЛЕКТРОННЫЕ РЕСУРСЫ

Для быстрого доступа к репозиторию и материалам лабораторной работы (исходный код `osmsmat.m`, аудиозаписи `voices.wav` и иллюстрации) ниже представлен QR-код (Рисунок 7):

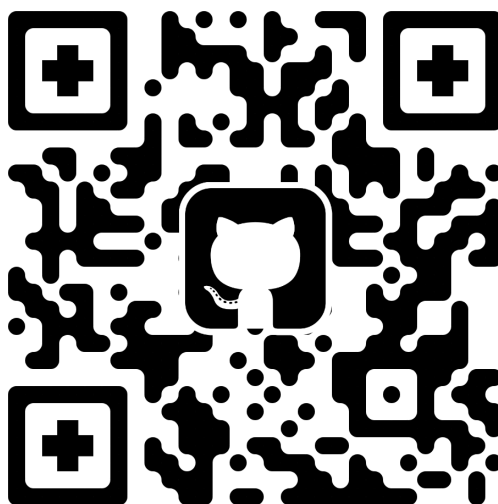


Рисунок 7 - QR-код для доступа к материалам и исходным кодам лабораторной работы